



# Bitwise operators

## One's and Two's complement



# Why One's and Two's complement

- Essential for performing arithmetic operations and ensuring consistency in mathematical operations
  - Representation of Negative Numbers
- Simplifies arithmetic operations
  - In two's complement, all arithmetic operations (addition, subtraction, multiplication and division) for both signed and unsigned numbers can be performed using same h/w
- Handling overflow
  - Two's complement arithmetic automatically handles overflow.

# One's complement

- Method for representing signed numbers
  - Positive numbers are represented as binary numbers
  - Negative numbers are represented as ones complement of the corresponding positive number

# Steps to find One's complement

- Steps to find the one's complement of a decimal number  $n$ 
  - Represent  $n$  in binary
  - Invert all the bits, that is 0's to 1's and vice versa
- Example: Suppose we're working with 8-bit quantities
  - Find one's complement of 7

7 in binary is: 0000 0111

One's complement of 7: 1111 1000

# Example: ones complement

- Representation of numbers in a one's complement machine

| Value of n | Binary representation of n | Bitwise complement | Value of -n |
|------------|----------------------------|--------------------|-------------|
| 7          | 0000 0111                  | 1111 1000          | -7          |
| 8          | 0000 1000                  | 1111 0111          | -8          |
| 9          | 0000 1001                  | 1111 0110          | -9          |
| -7         | 1111 1000                  | 0000 0111          | 7           |
| 0          | 0000 0000                  | 1111 1111          | -0          |

# Problem with ones complement

- Two representations of Zero
  - An all-zero bit pattern
  - An all-one bit pattern.
- This can lead to confusion and inconsistencies in arithmetic operations.

# Twos complement

- Method for representing signed integers in binary
  - Most common representation used in modern computer system
- Only one representation of zero
  - The all-zero bit pattern
  - This eliminates the ambiguity present in ones' complement.

# Steps to compute two's complement

- Steps to find the two's complement of a decimal number  $n$ 
  - Find the one's complement of  $n$
  - Add 1 to the one's complement
- Example: Suppose we're working with 8-bit quantities

- Find two's complement of 7

7 in binary is: 0000 0111

One's complement of 7: 1111 1000

Add 1:                   1

Two's complement of 7: 1111 1001

# Representing numbers in two's complement

- Representation a positive number  $n$ 
  - Bit string obtained by writing  $n$  in base 2
  - Example: 7 will be represented as  
7 will be written as 0000 0111
- Representation of negative number  $n$  (that is  $-n$ )
  - Find the two's complement of  $n$
  - Example: representation of  $-7$

| Value of $n$ | Binary representation | Bitwise complement | Two's complement | Value of $-n$ |
|--------------|-----------------------|--------------------|------------------|---------------|
| 7            | 0000 0111             | 1111 1000          | 1111 1001        | -7            |

any number starting with a 0 is positive, and any number starting with a 1 is negative.



## Some more examples

- Representation of numbers in a two's complement machine

| Value of n | Binary representation of n | Bitwise complement | Two's complement of -n | Value of -n |
|------------|----------------------------|--------------------|------------------------|-------------|
| 7          | 0000 0111                  | 1111 1000          | 1111 1001              | -7          |
| 8          | 0000 1000                  | 1111 0111          | 1111 1000              | -8          |
| 9          | 0000 1001                  | 1111 0110          | 1111 0111              | -9          |
| -7         | 1111 1001                  | 0000 0110          | 0000 0111              | 7           |
| 0          | 0000 0000                  | 1111 1111          | 0000 0000              | 0           |

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ (and you carry a 1 to the next column)}$$





# Bitwise operators

## Logical operators



# Bitwise logical operators



| Name                       | Symbol   |
|----------------------------|----------|
| (unary) bitwise complement | $\sim$   |
| bitwise AND                | $\&$     |
| bitwise exclusive OR       | $\wedge$ |
| bitwise inclusive OR       | $ $      |

*(Bitwise XOR)*

*1 if one or the other or both are 1.*

*1 if one or the other, but not both, are 1.*



# Bitwise complement

*If  $a = n$ , then  
 $\sim a = -(n+1)$ ; for all  $n$*



```
#include <stdio.h>
```

```
int main() {
```

```
    int num = 5; // binary: 0000 0101
```

```
    int result = ~num; // bitwise complement
```

```
    printf("Result: %d\n", result); // Output: -6 (binary: 1111 1010)
```

```
    return 0;
```

```
}
```

```
~
```



# Bitwise binary logical operators

- Truth table

| a | b | $a \& b$ | $a \wedge b$ | $a   b$ |
|---|---|----------|--------------|---------|
| 0 | 0 | 0        | 0            | 0       |
| 0 | 1 | 0        | 1            | 1       |
| 1 | 0 | 0        | 1            | 1       |
| 1 | 1 | 1        | 0            | 1       |

# Bitwise AND



```
#include <stdio.h>
```

```
int main() {
```

```
    int num1 = 5; // binary: 0000 0101
```

```
    int num2 = 7; // binary: 0000 0111
```

```
    int result = num1 & num2; // performs bitwise AND operation
```

```
    printf("Result: %d\n", result); // Output: 5 (binary: 0000 0101)
```

```
    return 0;
```

```
}
```



# Bitwise XOR



```
#include <stdio.h>
```

```
int main() {
```

```
    int num1 = 5; // binary: 0000 0101
```

```
    int num2 = 7; // binary: 0000 0111
```

```
    int result = num1 ^ num2; // performs bitwise AND operation
```

```
    printf("Result: %d\n", result); // Output: 2 (binary: 0000 0010)
```

```
    return 0;
```

```
}
```



# Bitwise OR



```
#include <stdio.h>
```

```
int main() {
```

```
    int num1 = 5; // binary: 0000 0101
```

```
    int num2 = 7; // binary: 0000 0111
```

```
    int result = num1 | num2; // performs bitwise AND operation
```

```
    printf("Result: %d\n", result); // Output: 7 (binary: 0000 0111)
```

```
    return 0;
```

```
}
```



# Exercise



- Write a C program to swap two numbers without using a temporary variable. Use bitwise operators to achieve this.
- STEPS
- Take two number num1 and num2
- Perform bitwise operation,  $\text{num1} \leftarrow \text{num1} \wedge \text{num2}$
- Perform bitwise operation,  $\text{num2} \leftarrow \text{num1} \wedge \text{num2}$
- Perform bitwise operation,  $\text{num1} \leftarrow \text{num1} \wedge \text{num2}$





# Bitwise operators

## Shift operators



# Left shift operators

- Left shift

`expr1 << expr2`

- `expr1` and `expr2` must be integral expressions
- causes bit representation of `expr1` shifted to left by `expr2` places

- Example

```
int num = 7; // 0000 0111
```

```
// Left shift by 2 positions
```

```
int shiftedNum = num << 2; // 0001 1100 (28)
```

`7 << 2 == 7 * 2^2 = 28`

*All 8 bits move two positions to the left.  
The two leftmost 0's fall off and are discarded.  
Two new 0's are added on the right to fill the gaps.*

Original: 0 0 0 0 0 1 1 1

| |

(these fall off)

Shifted: 0 0 0 1 1 1 \_ \_

| |

**Trick :  $a \ll n = a * 2^n$**

(add 0s)

Result: 0 0 0 1 1 1 0 0

# Right shift operators

- Right shift  
`expr1 >> expr2`
  - `expr1` and `expr2` must be integral expressions
  - causes bit representation of `expr1` shifted to right by `expr2` places
- Example  
`int num = 7; // 0000 0111`  
`//Right shift by 2 positions`  
`int shiftedNum = num >> 2; // 0000 0001 (1)`

# Review

1. Left shift: `int x = 12; printf("%d", x << 2);`

- What is the output and what is it equivalent to?  $12 * 2^2 = 48$

2. Right shift: `int x = 12; printf("%d", x >> 2);`

- What is the output and what is it equivalent to?  $12 / 2^2 = 3$

3. `x & (-x);` What is this equivalent to?

4. `int a = 7, b = 4;`

`printf("%d", a | b);`

7 -> 0000 0111  
4 -> 0000 0100  
7 | 4 -> 0000 0111 = 7

4  
A number that isolates the rightmost '1' bit of x.

12 -> 0000 1100  
-12 -> 1111 0100

# Exercise

- Given two integers a and b, determine how many bits are different between them — i.e., how many bit positions must be flipped in a to make it equal to b.
  - Write the function, `int countFlips(int a, int b);`
  - Find the difference in their bit position; that is if the two bits in the same position is different set to 1
  - Count the number of 1's.